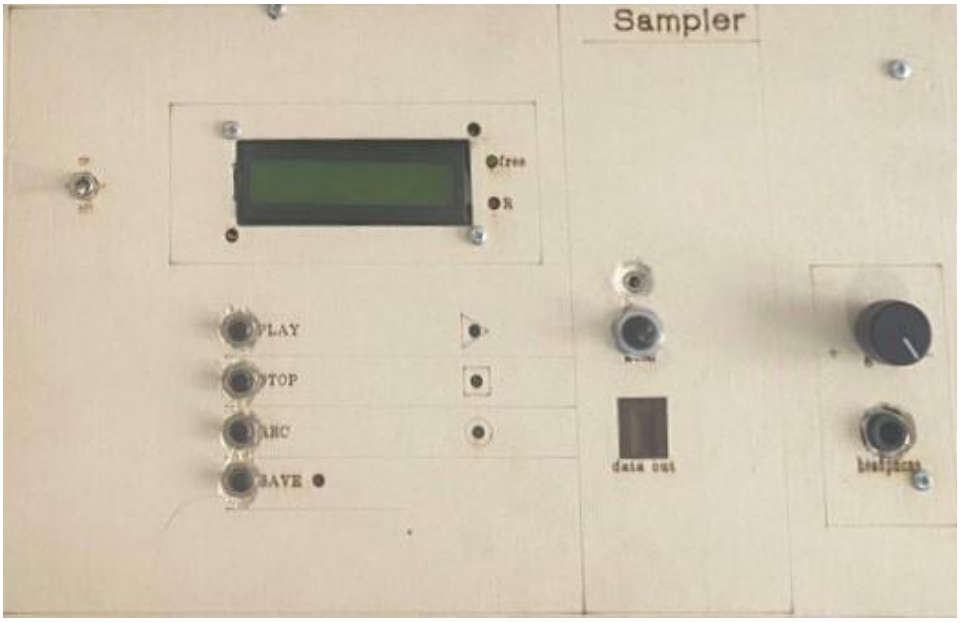
3.8) CAMPIONATORE:

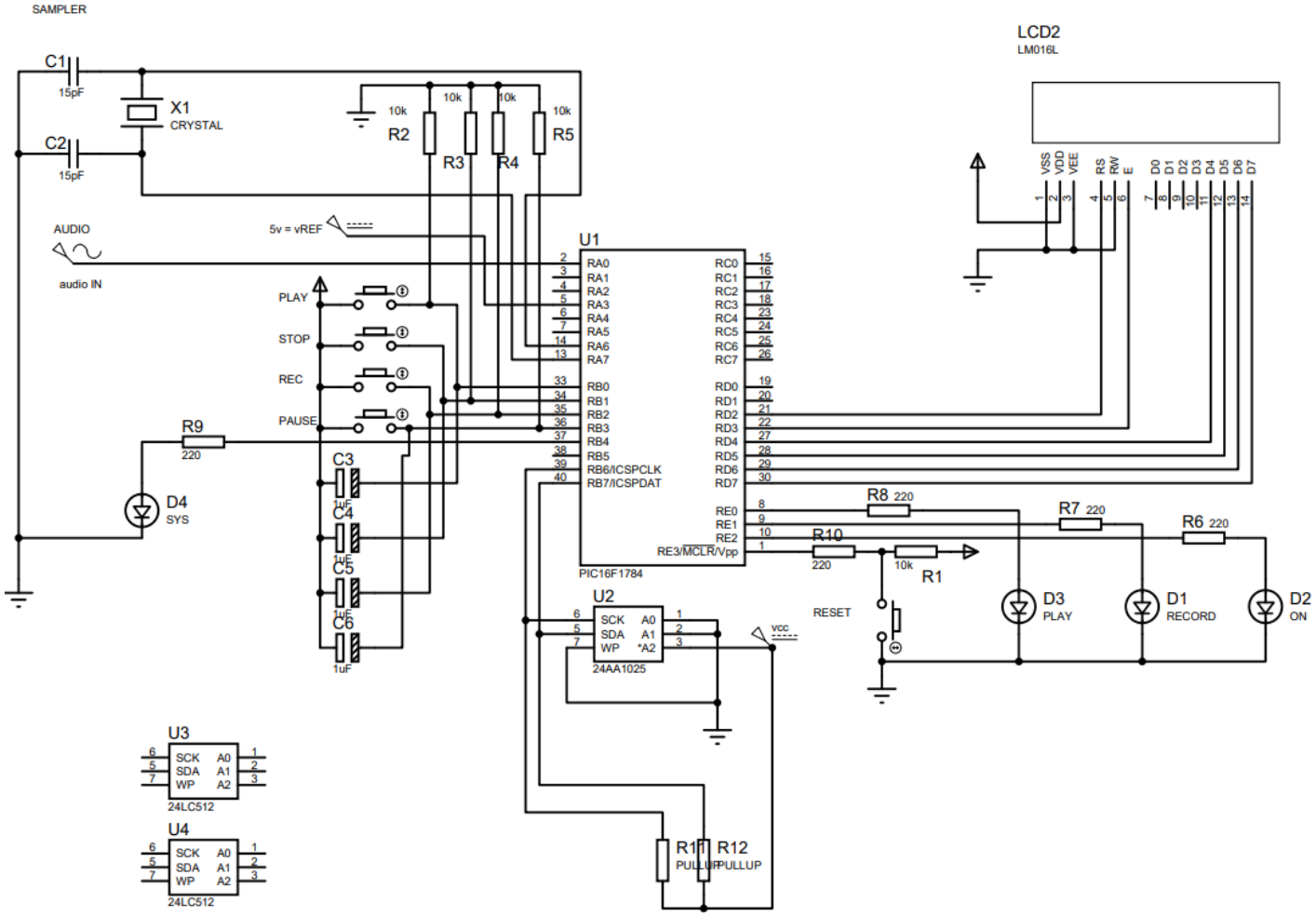
Può essere molto utile poter registrare ciò che viene suonato, fornendo la possibilità di salvare in memoria una propria melodia o idea che, arrangiata su ESynth, possa essere riascoltata. Per questo motivo è stato progettato un campionatore a 12 bits / Fc=50 KHz, con l'utilizzo di un microcontrollore: PIC16F1784 e di una memoria eeprom da 1 MByte: 24LC1025.
Viene utilizzato l'ADC a 12 bits disponibile sul uC; il salvataggio dei dati campionati, sulla eeprom, avviene tramite protocollo I2C; è possibile registrare fino a 10 secondi di audio. L'utilizzatore può accendere e spegnere il modulo di campionamento, regolare il volume dell'output (jack 2.5mm) e muoversi nel menù con i pulsanti tra le varie modalità disponibili (REC - PLAY - STOP). E' stato predisposto un pulsante "save" ed un' uscita "data out" per poter implementare in futuro la possibilità di salvare su chiavetta, in file .wav ("wave"), la registrazione presente sulla eeprom.



↳ interfaccia del campionatore, con uscita cuffia (a destra)

La eeprom 24LC1025 non era disponibile sul mercato, sono state utilizzate due eeprom 24LC512 (512 kByte), connesse in parallelo all' I2C. Purtroppo queste hanno delle tempistiche di scrittura troppo alte, potendo scrivere una "pagina" (128 Byte) in 5ms. Per questo motivo non è stato possibile far funzionare correttamente il campionatore. Il pic è stato programmato per poter eseguire la funzione di menù, con pulsanti, leds ed lcd esterni. Tutto è predisposto per essere completato e reso funzionante come da progetto, appena disponibile una eeprom da 1 Mbyte "sufficientemente veloce".

SCHEMA CIRCUITALE DEL CAMPIONATORE:



L'alimentazione a 5V è stata ottenuta con un integrato regolatore di tensione LDO: LP2957

LP2957/LP2957A 5V Low-Dropout Regulator for µP Applications

Check for Samples: LP2957, LP2957A

FEATURES

- **5V Output within 1.4% Over Temperature (A Grade)**
- **Easily Programmed for Snap-On/Snap-Off Output**
- **Ensured 250 mA Output Current**
- **Extremely Low Quiescent Current**
- **Low Input-Output Voltage Required for Regulation**
- **Reverse Battery Protection**
- **Extremely Tight Line and Load Regulation**
- **Very Low Temperature Coefficient**
- **Current and Thermal Limiting**
- **Error Flag Signals when Output is out of Regulation**

APPLICATIONS

- **High-Efficiency Linear Regulator**
- **Battery-Powered Regulator**

DESCRIPTION

The LP2957 is a 5V micropower voltage regulator with electronic shutdown, error flag, very low quiescent current (150 µA typical at 1 mA load), and very low dropout voltage (470 mV typical at 250 mA load current).

Output can be wired for snap-on/snap-off operation to eliminate transition voltage states where µP operation may be unpredictable.

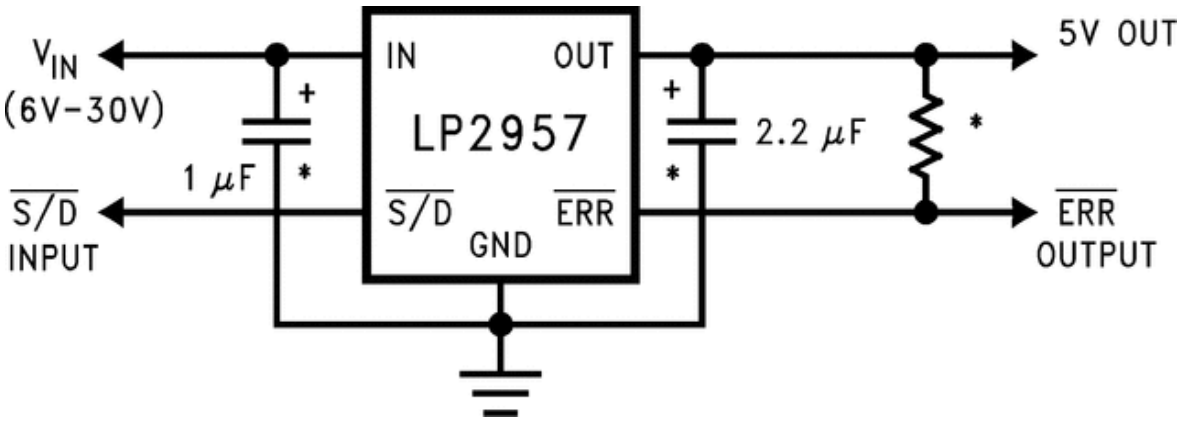
Output crowbar (50 mA typical pull-down current) will bring down the output quickly when the regulator snaps off or when the shutdown function is activated.

The part has tight line and load regulation (0.04% typical) and low output temperature coefficient (20 ppm/°C typical).

The accuracy of the 5V output is ensured at room temperature and over the full operating temperature range.

The LP2957 is available in the five-lead TO-220 and DDPAK/TO-263 packages.

SCHEMA CIRCUITALE DI COLLEGAMENTO:



4. PARTE QUARTA

SOFTWARE CAMPIONATORE

Sono stati settati i registri relativi all'ADC (12 bits) e DAC (8 bits) del PIC16F1784 ed attivati due interrupt :

- un primo da 1 ms, per il "timing del sistema", utilizzato al posto delle funzioni di "delay"
- un secondo da 20 uS, necessario per ottenere una frequenza di campionamento di 50 kHz

Quando il uC entra in modalità di campionamento, ogni 20uS viene acquisito il valore digitale (da 0 a 4096) della tensione in input e salvato su due byte della eeprom tramite I2C.
In modalità "play" vengono letti i valori della eeprom, due ogni 20 uS, portati in output analogico tramite il DAC.

.
.
.

DI SEGUITO IL LISTATO DEL PROGRAMMA CARICATO NEL uC:

```
/*******************************************************/
/******************CAMPIONATORE AUDIO*******************/
/*******************************************************/

//DICHIARAZIONE VARIABILI GLOBALI
unsigned long x;
unsigned long a= 0;
short mod = 0;
short indice = 0;
unsigned short adh[128];
unsigned short adl[128];
short i= 0;

// connessioni modulo lcd [PORTD]
sbit LCD_RS at RD2_bit;
sbit LCD_EN at RD3_bit;
sbit LCD_D4 at RD4_bit;
sbit LCD_D5 at RD5_bit;
sbit LCD_D6 at RD6_bit;
sbit LCD_D7 at RD7_bit;

sbit LCD_RS_Direction at TRISD2_bit;
sbit LCD_EN_Direction at TRISD3_bit;
sbit LCD_D4_Direction at TRISD4_bit;
sbit LCD_D5_Direction at TRISD5_bit;
sbit LCD_D6_Direction at TRISD6_bit;
sbit LCD_D7_Direction at TRISD7_bit;

  void campionatore()
  {
                      I2C1_Start();
   if(a<50000){
   I2C1_Wr(0b10100000);
   }else
   {
   I2C1_Wr(0b10100010);
   }
   I2C1_Wr(a>>8);
   I2C1_Wr(a);
   I2C1_Wr(ADRESH);
   I2C1_Wr(ADRESL);
                    I2C1_Stop();
  }

  void diaci()
  {
                    I2C1_Start();
         I2C1_Wr(0b10100001);
         I2C1_Wr(a>>8);
         I2C1_Wr(a);
         DACCON1 = I2C1_Rd(0);
         indice = indice+1;
         if (indice == 32){
           indice = 0;
                  I2C1_Stop();
         }
  }

void main()
{

    //DICHIARAZIONE VARIABILI LOCALI
```

```
63:     short valore;
64:     float audio;
65:     char volts[8];
66:
67:     //PULIZIA PORTs
68:     PORTA = 0;
69:     PORTB = 0;
70:     PORTC = 0;
71:     PORTD = 0;
72:     PORTE = 0;
73:
74:     //SETTAGGI PORTs
75:     C1ON_bit = 0;              //COMPARATORI DISABILITATI
76:     C2ON_bit = 0;              //COMPARATORI DISABILITATI
77:     ANSELA.F0 = 1;             //RA0 ANALOGICO
78:     ANSELD = 0;                //RD0-RD2 DIGITALI
79:     ANSELB = 0;                //PORTB DIGITALE
80:     ANSELE = 0;                //PORTE DIGITALE
81:     //APFCON = 0x18;
82:     TRISA = 0b00000000;        //RA0 DI INPUT
83:     TRISB = 0b00001111;        //INPUT PULSANTI [RB0 - RB3] + LED sys [RB4]
84:     TRISC = 0x00;
85:     TRISD = 0b00000011;        //CONNESSIONI LCD
86:     TRISE = 0b00001000;        //MCLR + LEDS PLAY - REC - ON
87:     PORTB.F4 = 0;              //LED sys
88:     PORTE.F2 = 1;              //LED ON
89:
90:     // Configurazione dell'ADC
91:     ADCON0 = 0b00000000;  //12bit, an0 input, godone=0, an0=0
92:     ADCON1 = 0b00100001; //sign magnitude, Fosc/32 (per Tad = 1.6uS), vref- a mass
    sa, vref+ a AN3
93:     ADCON2 = 0b00000001;
94:
95:     // Configurazione del DAC
96:     FVRCON  = 0b00000000;
97:     DACCON0 = 0b00100000;
98:     DACCON1 = 0;
99:
100:    //SETTAGGI INTERRUPT DA 1ms(Tsys)
101:    OPTION_REG = 0x84;
102:    TMR0 = 6;
103:    INTCON = 0xE0;
104:
105:    //SETTAGGI INTERRUPT DA 20uS(Tadc)
106:    T1CON          = 0x00;
107:    TMR1IF_bit          = 0;
108:    TMR1H          = 0xFF;
109:    TMR1L          = 0xA9;
110:    TMR1IE_bit          = 1;
111:    INTCON          = 0xE0;
112:
113:    //INIZIALIZZAZIONI E "INTRO" ALL'ACCENSIONE
114:    I2C1_Init(400000);
115:
116:    Lcd_Init();
117:    Lcd_Cmd(_LCD_CLEAR);
118:    Lcd_Cmd(_LCD_CURSOR_OFF);
119:    Lcd_out(1,1,"***SYNTH-V1.0***");
120:    Lcd_out(2,1,"................");
121:    delay_ms(2000);
122:    Lcd_Cmd(_LCD_CLEAR);
123:    Lcd_out(1,1,"**CAMPIONATORE**");
```

```
124:     Lcd_out(2,1,"PLAY-RECORD-SAVE");
125:
126:     //SETTAGGI VARIABILI
127:     x = 0;
128:
129:     while(1)
130:     {
131:                     if(a>=500000){
132:                     T1CON          = 0x00;
133:                          Lcd_out(2,1,"FINISH          ");
134:                          delay_ms(700);
135:                          Lcd_out(2,1,"               ");
136:                          delay_ms(600);
137:                          Lcd_out(2,1,"FINISH          ");
138:                          delay_ms(700);
139:                          Lcd_out(1,1,"**CAMPIONATORE**");
140:                          Lcd_out(2,1,"PLAY-RECORD-SAVE");
141:
142:                          a = 0;
143:                          mod = 0;
144:                          PORTE.F0 = 0;
145:                          PORTE.F1 = 0;
146:                     }else if(a>=450000){
147:                          Lcd_out(2,1,"9");
148:                     }else if(a>=400000){
149:                          Lcd_out(2,1,"8");
150:                     }else if(a>=350000){
151:                          Lcd_out(2,1,"7");
152:                     }else if(a>=300000){
153:                          Lcd_out(2,1,"6");
154:                     }else if(a>=250000){
155:                          Lcd_out(2,1,"5");
156:                     }else if(a>=200000){
157:                          Lcd_out(2,1,"4");
158:                     }else if(a>=150000){
159:                          Lcd_out(2,1,"3");
160:                     }else if(a>=100000){
161:                          Lcd_out(2,1,"2");
162:                     }else if(a>50000){
163:                          Lcd_out(2,1,"1");
164:                     }
165:               //PULSANTI E MODALITA':
166:               if (Button(&PORTB, 0, 30, 1)){  //PLAY BUTTON
167:                    ADON_bit = 0;
168:                    mod = 1;
169:                    PORTE.F0 = 1;
170:                    PORTE.F1 = 0;
171:                    Lcd_out(1,1,"***SYNTH-V1.0***");
172:                    Lcd_out(2,1,"0S      --PLAY  ");
173:                    DACCON0 = 0b10100000;
174:                    T1CON          = 0x01;
175:                    diaci();
176:               }
177:               if (Button(&PORTB, 1, 150, 1)){  //STOP BUTTON
178:                    ADON_bit = 0;
179:                    mod = 2;
180:                    PORTE.F0 = 0;
181:                    PORTE.F1 = 0;
182:                    T1CON          = 0x00;
183:                        Lcd_out(1,1,"**CAMPIONATORE**");
184:                        Lcd_out(2,1,"PLAY-RECORD-SAVE");
185:                        a = 0;
```

```
186:           }
187:           if (Button(&PORTB, 2, 3000, 1))  //REC BUTTON
188:           {
189:                ADON_bit = 1;
190:                delay_us(30);
191:                PORTE.F0 = 0;
192:                PORTE.F1 = 1;
193:
194:                Lcd_out(1,1,"***SYNTH-V1.0***");
195:                Lcd_out(2,1,"0S      --RECORD");
196:                mod = 3;
197:                T1CON         = 0x01;                      // attiva timer 1
198:                a = 0;        //reset tempo
199:                campionatore();
200:
201:            }
202:
203:      //1sec. sys -> LED BLINK
204:           if (x >= 1000)
205:           {
206:                PORTB.F4 = ~PORTB.F4 ;
207:                x = 0;
208:                TMR0IF_bit = 0;
209:           }
210:
211:      }
212: }
213:
214: //l'interrupt avviene ogni ms e ogni secondo x si azzera e fa blinkare il led pe
      un secondo
215: void Interrupt(){
216:      if (TMR1IF_bit){
217:
218:          TMR1IF_bit = 0;
219:
220:          TMR1H         = 0xFF;
221:          TMR1L         = 0xA9;
222:
223:          adh[indice] = ADRESH;
224:          adl[indice] = ADRESL;
225:          indice++;
226:          if(indice == 127){
227:           indice = 0;
228:          }
229:          a++;
230:
231:      }else if (TMR0IF_bit){
232:          TMR0IF_bit = 0;
233:          TMR0 = 100;
234:          x++;
235:      }
236: }
```